MCP TRIOLOGY

Model Context Protocols

Claude provides better integration with MCP servers

Story of MCP:

LLM enabled a productive boom and then we saw LLM API revolution with tools like co-pilot.
Thus LLM become more accessible and most of the software/platform became AI enabled.

Acessibility caused a problem where AI tools of different platforms dont know what other AI think and how do they work.
Thus we were trying to solve this problem using an unified AI agent/tool

Context: Major Problem of Unified AI

Context means all the information that LLM uses to generate a response. It simply means all those parameters,knowledge base,conversation history that is being used by LLM to generate content

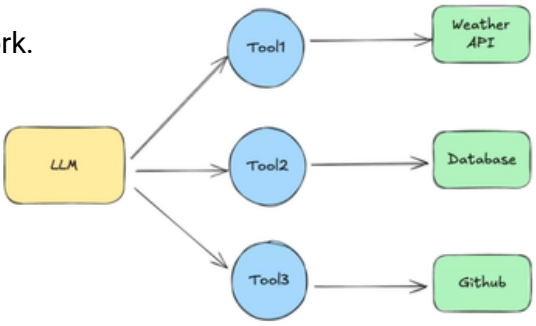
Eg: In an application you can have different context for different tools like database,github,Drive etc because a prompt to LLM model may require giving context of all this which should include { DB schema,Drive folder content,gihub repo } which may become quite hectic .

For bigger applications this copy paste method for different contexts would fail

SOLUTION **Function calling, Tools for different task**

Here, LLM activates different tools for different context and work.
This can be considered as tool augmented Architecture.

Now LLM will interact with tools and reduce manual work.
For LLM, user acts like human API providing prompt



Eg: Google connectors,Database query tool,salesforce integration

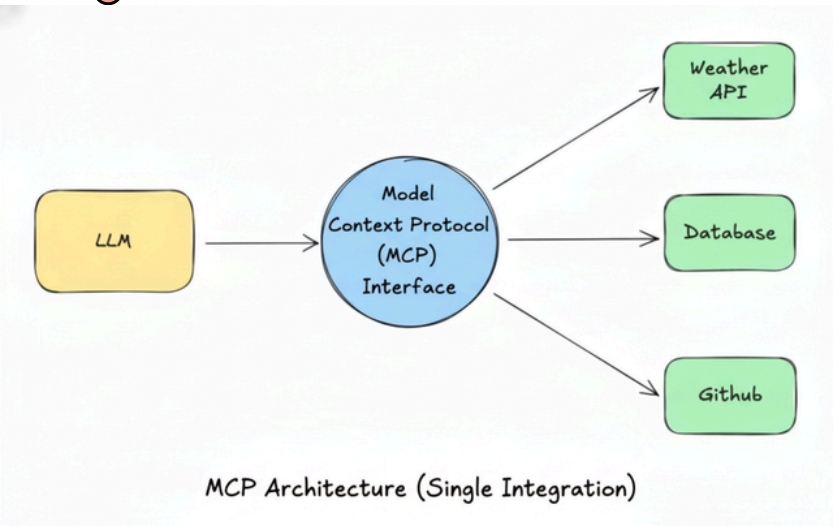
Disadvantage with tool augmented Architecture

- **Integration Problem:** N*M integrations (N = no. of chatbot,M=tools)
- **Maintenance Problem:** Each integration requires authentication,API gateways,data format consistency.
- **Cost & Time:** For every chatbot we need to perform individual integration and need to hire workforce to manage this complex integrations.

SOLUTION **Single Integration (MCP)**

Notes : Rabindra Nehru Mishra

MCP (Model Context Protocol)



MCP Architecture (Single Integration)

MCP has a client(chatbot) and server(services/tool).
There can be mutliple servers

Language of Communication between client and server is known as MCP.

How do I make Chatbot as MCP client & Tools as MCP server?

MCP client SDK,MCP Server SDK

Function Calling (API) V/S MCP

In MCP, server does heavyLifting work and we dont need to do anything from client side.In MCP,client just have to simply connect unlike tool calling.

Server

```python
@app.route("/weather")
def weather_endpoint():
    city = request.args.get("city")
    # Query internal weather database
    result = lookup_weather(city)   #
    return jsonify(result)
```

Client

```python
def get_weather(city: str):
    url = f"https://myweatherapi.com/weather?city={city}&key=API_KEY"
    response = http_get(url)
    data = json_parse(response)
    return f"{city}: {data['temp_c']}°C, {data['condition']}"
```

Here, in function calling we need to manage client side (code snippet)

Famous service providers like Github,Drive built their MCP servers so that any MCP client (AI chatbot/Application) can connect with that server.

If API changes,it is the MCP server responsibility to handle all the connections as in client side we simply have to connect with the client.Thus no maintenance overhead.

At Day1 your AI applications can connect with numerous MCP servers without integration complexity thus reducing cost and time.

'MCP server ecosystem will grow with rising AI clients'

Architecture of MCP:

1) Simplest Architecture

Consist a simple host and server where host are nothing but AI chatbot.Behind the scene host could be connected to any custom or self made LLM.
Server can be considered as a tool like github,G-Drive,Slack etc.

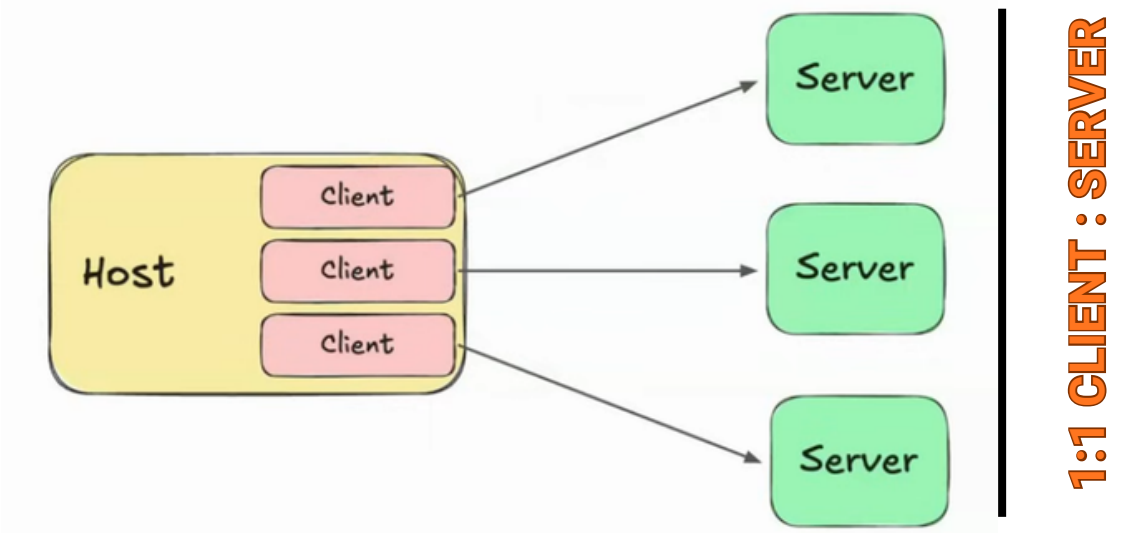
Eg:- User sends a request to Host asking whether there are any new commits on github repo or not? Host forward this prompt to LLM. However LLM dont have context of github repo so they ask Host to get this information with connected mcp server of github and then host returns the context to LLM and LLM generates the response and then host revert the response to the user.

In Reality Host doesnt communicate with Server directly

Host communicates with server via MCP client. Host forwards high level request to mcp client , then mcp client converts it to mcp compatable request and sends it to mcp server and after this mcp server generates a structured mcp response which is then sent back to mcp client upon which mcp client translates that response so that Host could understand that.

Note: There is 1:1 relationship between client and server

We need to have multiple clients for different mcp servers and here client acts as a helper device for host. All low level communication is done by MCP client and server. Their is nothing to do with the host.



Advantages

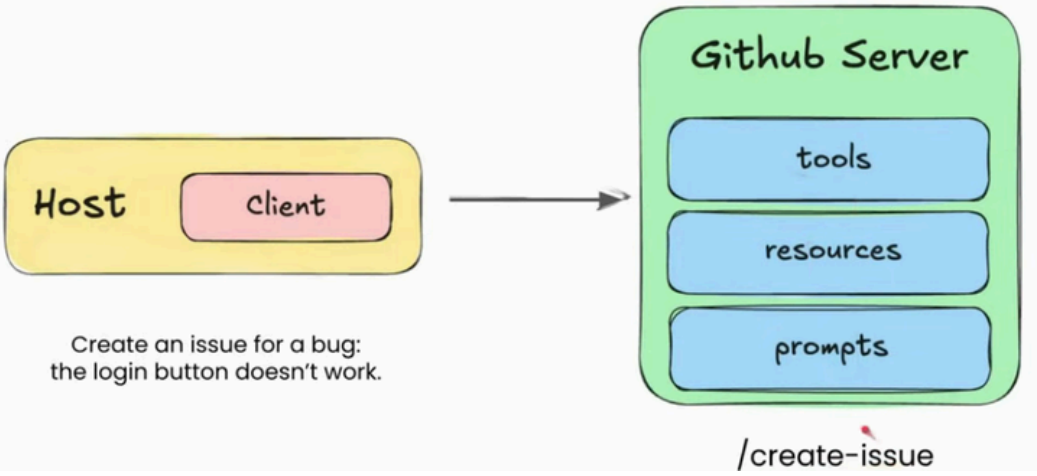
- Decoupled (Host is not strongly connected with server)
- Parallelism
- Scalability (one host can connect to various servers)

Primitives:

Things/Services that MCP server offers to Host.

1. Tools:- Actions that servers provide.(eg telling no. of active issues).
2. Resources:-Structured data sources allowing clients to read from. (eg Readme File)
3. Prompts:-Predefined Prompt Template [Guidelines]

AI chatbot could pass vague prompt which not necessarily be understood by other mcp server.
Using Prompt primitives AI Host can frame prompt specific to server template.



Prompt Primitive eg:

Prompt perceived by server

```
{
  "name": "issue_report_prompt",

  "description": "Write clear, detailed GitHub issues",

  "messages": [{
      "role": "system",

      "content": "Always include: Title, Steps to
Reproduce, Expected, Actual, Environment"}
  }]
}
```

Prompt from client side

```
Title: Bug – Login Button Not Working

**Steps to Reproduce:**
1. Open the login page
2. Enter valid credentials
3. Click the Login button

**Expected Behavior:**
User should be logged in and redirected to dashboard.

**Actual Behavior:**
Nothing happens when clicking the Login button.

**Environment:**
Chrome 121, macOS 14.2
```

MCP server also provides standard operations with primitives

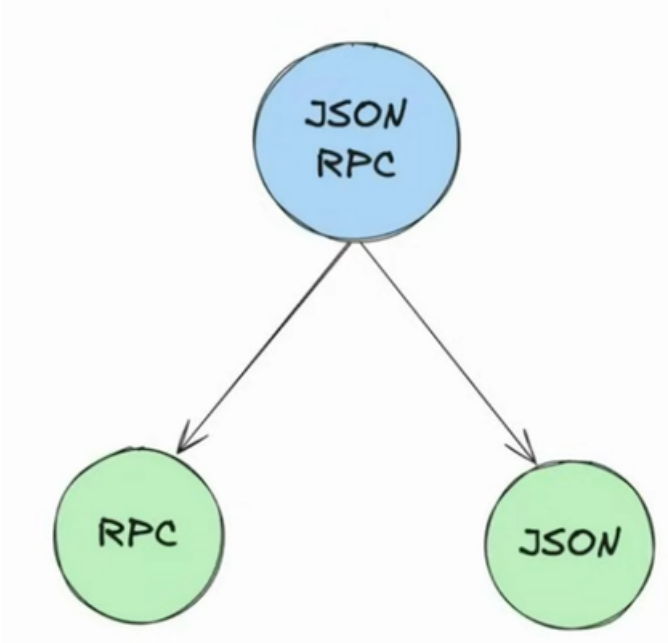
Data Layer of MCP

Language and grammar that is followed in MCP ecosystem to communicate.
JSON RPC-2.0 serves as foundation of data layer.

JSON RPC -2.0 (Remote Procedure Call)

Understand RPC

1. Here,2.0 simply represents version of current JSON RPC.
2. RPC is mainly used in distributed computing.It executes function/procedure on another computer as if it feels like running it on local ,while hiding details of network communication and data transfer.
3. RPC= remote call of procedure
4. Distributed system means different package dependencies/components residing on different servers and one server executing procedures feeling like running it on local.



JSON RPC is marriage between JSON & RPC.
Here, RPC response would be in JSON format and function call response will also be in JSON format.

Undesrstand Request Structutre

RPC request in JSON

```
{
  "jsonrpc": "2.0",
  "method": "add",
  "params": [2, 3],
  "id": 1
}
```

RPC response in JSON

```
{
  "jsonrpc": "2.0",
  "result": 5,
  "id": 1
}
```

RPC error response in JSON

```
{
  "jsonrpc": "2.0",
  "error": { "code": -32601, "message": "Method not found" },
  "id": 1
}
```

"id" is used for mapping request and response. Suppose,if method add is not found it will also generate an error response that too in json format.

Note: For every RPC request there has to be response

Batching: Multiple calls(method) in one request

```
[
  {
    "jsonrpc": "2.0",
    "id": 5,
    "method": "tools/call",
    "params": { "name": "github.listIssues", "arguments": { "owner": "owner",
  },
  {
    "jsonrpc": "2.0",
    "id": 6,
    "method": "tools/call",
    "params": {"name": "github.listPulls", "arguments": { "owner": "owner",
  }
```

Notification: Fire & Forget

Unlike normal requests in case of notifications server is not responsible for sending a reply thus name fire and forget. Here it is not compulsion that every notification must have a response

6) Notification (fire-and-forget)

Notification (server → client)

```json
{
  "jsonrpc": "2.0",
  "method": "files/updated",
  "params": {
    "fileId": "abc123",
    "name": "ProjectPlan.docx",
    "updatedBy": "alice@example.com",
    "timestamp": "2025-09-12T08:15:30Z"
  }
}
```

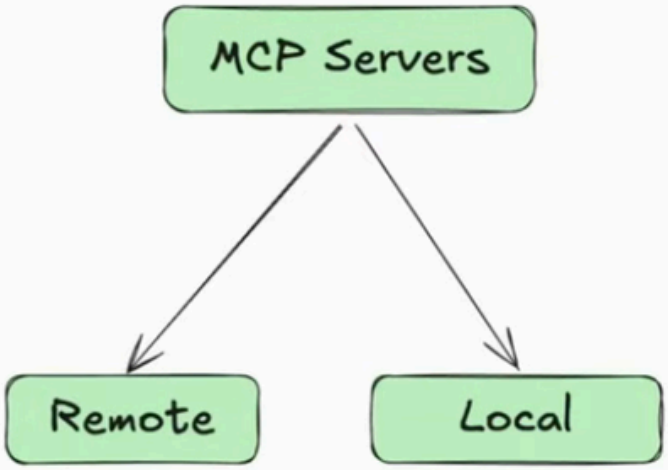
Here, server has send notification to client regarding update.
Client not necessarily need to do anything with that message.
Here, we also dont have id parameter because response is not compulsory.

Why JSON RPC instead of REST??

1. Unlike REST RPC requests are lightweight as they dont necessarily include headers.
2. REST uses HTTP while RPC works well with STDIO/HTTP/SSE
3. Transport Agnostic.
4. Supports Batching and Notification
5. In REST one request per call is allowed no scope of batching.
6. RPC are bi-directional, even servers can send requests.

Transport Layer

Mechanism that moves JSON-RPC messages between client & server



A **local server** is a program running on your own computer.

-> STDIO

A **remote server** is a program running on another computer (somewhere else on the network or internet) that you connect to over a network.

-> HTTP/SSE

Local servers can locate or find local files to get context

eg:- finding a .py file from local machine.

For Local servers ------>STDIO Transport Protocol

For Remote servers ---------> HTTP/SSE protocol

The host launches the server as a subprocess on the same machine.

The host(client) writes JSON-RPC messages into the server's STDIN

The server reads those messages, processes them, and writes back responses to it's STDOUT

Benefits of the STDIO transport

Fast - data is passed directly between processes.

Secure - no open network port that could be attacked; communication is only local.

Simple → every language/runtime supports reading/writing from stdin/stdout. No extra libraries required.

Remote Servers - HTTP + SSE

Using HTTP allows the host to reach Servers running anywhere

Host sends JSON RPC requests using POST requests with a JSON payload

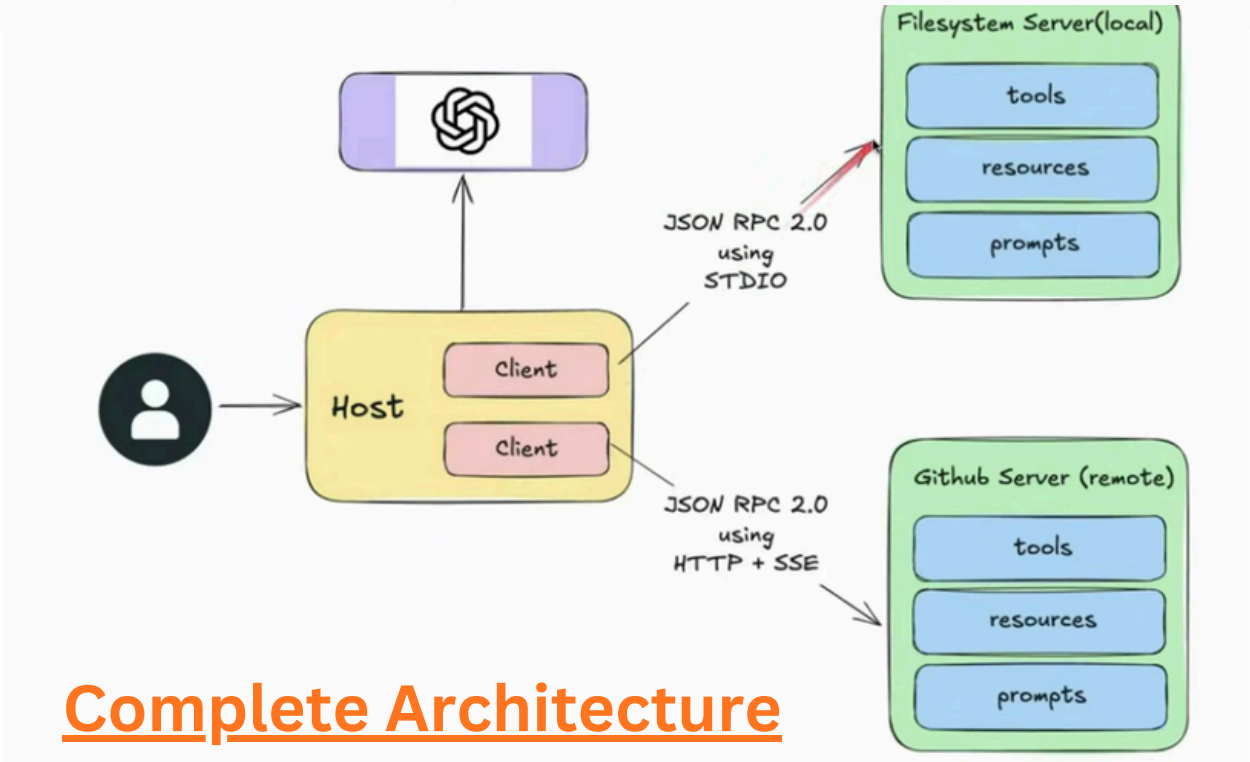
The transport supports standard HTTP auth methods(like API keys)

SSE stands for Server Sent Events, and it's an extension of HTTP

Using SSE the server sends multiple messages to client over a single open connection

Instead of sending one large JSON blob, the server can stream chunks of data as they are ready

Ideal for long running tasks or incremental updates



Complete Architecture

MCP Lifecycle

MCP life cycle describes sequence of steps that govern how a host and a server establish,use and enda a connection.

3 Stages of MCP Lifecycle

1. Initialization
2. Operation
3. Shut Down (closing our client)

Initialization

First interaction between client and server must occur in this stage. Always ensure protocol version compatibility.
Here client server Exchange and negotiate capabilities.

Client sends a initialize request containg (step 1)

[Note: Other than ping requests here client cannot send anything else like tool request etc]

a. method : “initialize”
b. protocolVersion: “2025-03-26”
c, capabilities ;{ “sampling:{},”roots”:{”listChanged”:true}
d. “clientInfo”:{”name”:”IDEPlugin”,”version”:”1.0.0"}
// Basically by sending capabilities client is telling the server what all functionalities client can support or give.

Server sends its own capabilties and info (step 2)

a. serverInfo:{”name”: {},”version”:{}}
b.protocolVersion:”2025-03-26"
c. capabilities:{}

Initialized Notification (step 3)

[Note: server can only send pings and logging requests before receiving initialized notification]

After sucessfull initialization client must send a notification which is also known as initialized notification.

eg: {
“jsonrpc”:”2.0",
“method”:”notifications/initialized”
}

Now the client and Server are connected

Understanding Negotiation Capability

Here , client server both understand about what they can demand/ask from their capabilities

Client : roots
Client : sampling
client : elicitation

1.roots capability: Client gives acess of its base directory(file system) to Server.
2.Sampling capability: Bi-directional request. Server can also ask for help to client if it has some capabilities.
3.Elicitation: Drawing out, extracting, or bringing forth information, a reaction, or a response from client (in case client didnt gave enough information to server)

Server : prompts
Server : resources (static / context)
Server : tools
Server : logging (usually in long running tasks server logs the logging state)

Sub Capabilities

1. Listchanged: during the session suppose a new tool gets added
2. Subscribe: if a particular resources gets change we give notification from server to client

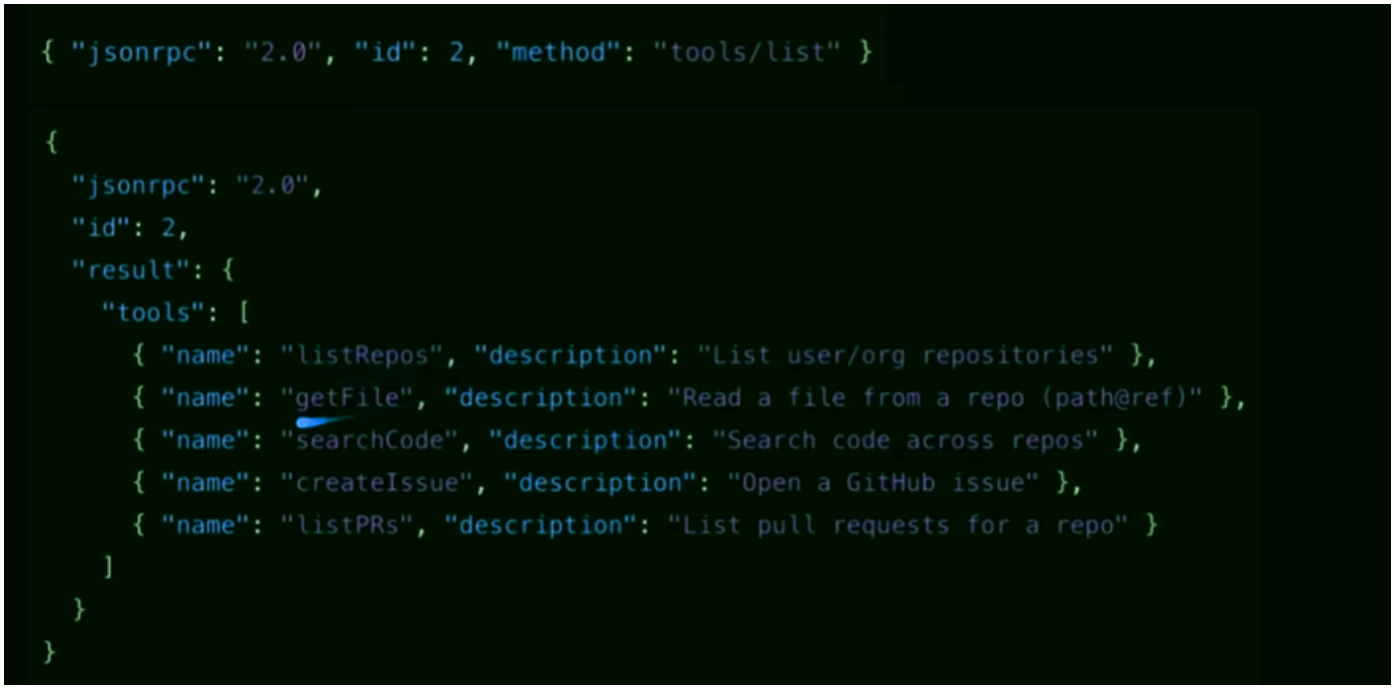
Operation Phase

During the operation phase the client and the server exchange messages according to the negotiated capabilities.

In operation phase respect the negotiated protocol version and only use capabilities that were sucessfully negotiated.

Capability Discovery

In Initialization phase client knows about what all tools the server has but doesnt know about the specific capabilities of those tools.
Thus , here client tries to learn about tools capabilities that server has by sending the request



Tool Calling

Here,it tries to understand user request and try to map it to appropriate tool.
This is second step of operational phase

Shutdown Phase

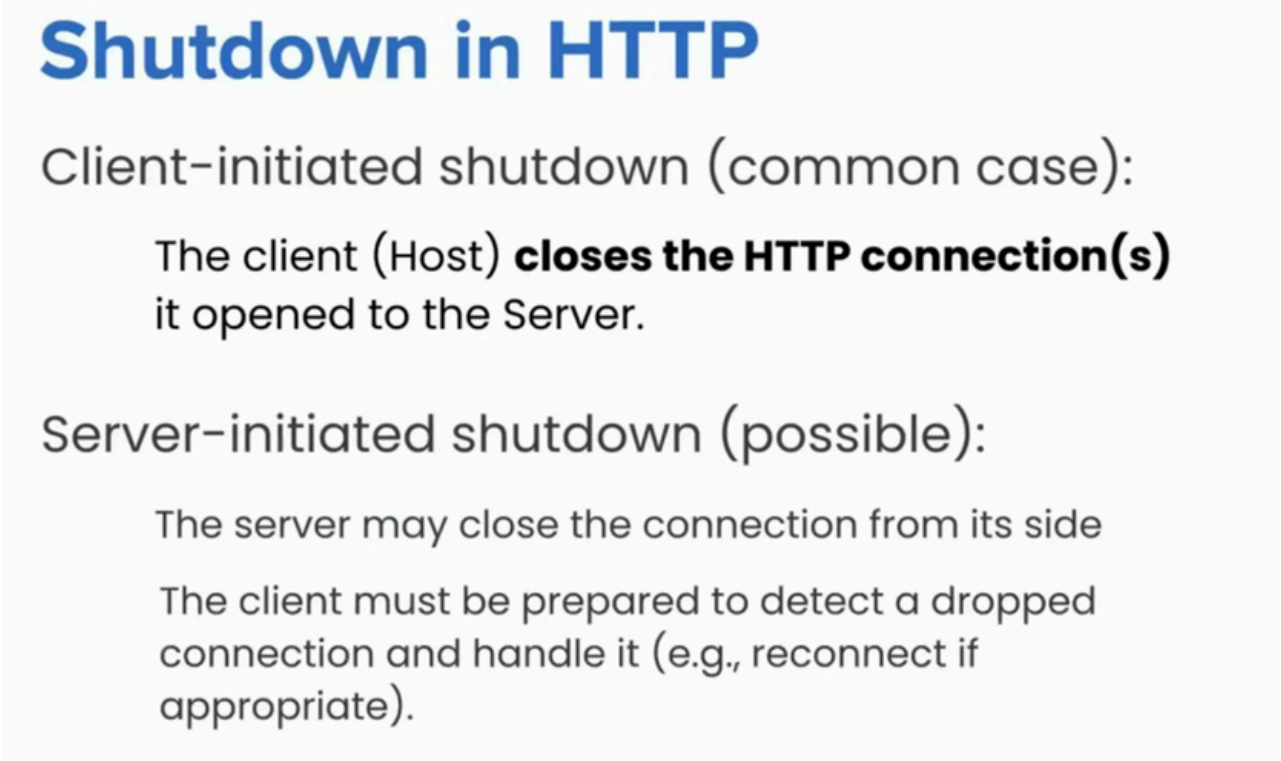
Generally one side initiates the shutdown (usually client), either client gets shutdown or server gets shutdown.
Here , we dont se any json rpc messages gettting communicated instead during shutdown this responsibility lies with transport layer for signalling termination.

In case of Local (stdio)

SIGTERM = (Signal Terminate) client can take help from OS to send SIGTERM to terminate server.
SIGKILL = (signal kill) low level request to OS to kill the server

In nutshell , client stops its input stream and waits for the server to get killed or server stops its output stream and exits the process.

If in case it doesnt happen then client can make use of SIGTERM or SIGKILL.



In case of Server initiated shutdown client should be prepared to handle the shutdown properly. Because need to analyze why server failed due to anamoly or as a shutdown process.

Special Case

Pings

Ping is a lightweight request/response method defined in MCP.

Purpose: to check whether the other side (Host or Server) is still alive and the connection is responsive.

```
{ "jsonrpc": "2.0", "id": 42, "method": "ping" }
```

```
{ "jsonrpc": "2.0", "id": 42, "result": {} }
```

Ping: Usually used in case of long running task to check whether connection is alive or not.

1. useful for checking if the other side is up before full initialize.
2. Prevents the connection from being dropped silently by OS, firewall or proxies.

Error Handling

Error handling in MCP is how the Host (client) and Server signal that **something went wrong** with a request

MCP inherits JSON RPC's standard error object format

Causes of Error

Unsupported or mismatched protocol version.

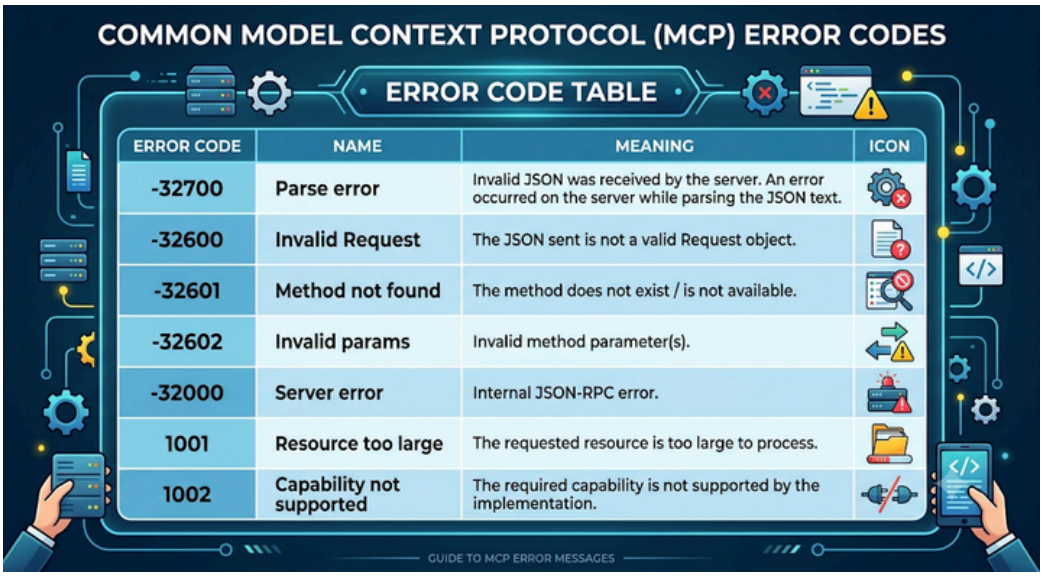
Calling a method for a capability that wasn't negotiated.

Invalid arguments to a tool

Internal server failure while processing a request.

Timeout exceeded → client cancels request.

Malformed JSON-RPC messages.



ERROR CODE	NAME	MEANING	ICON
-32700	Parse error	Invalid JSON was received by the server. An error occurred on the server while parsing the JSON text.	
-32600	Invalid Request	The JSON sent is not a valid Request object.	
-32601	Method not found	The method does not exist / is not available.	
-32602	Invalid params	Invalid method parameter(s).	
-32000	Server error	Internal JSON-RPC error.	
1001	Resource too large	The requested resource is too large to process.	
1002	Capability not supported	The required capability is not supported by the implementation.	

Timeout

Timeout is about ensuring requests don't hang forever

Purpose

Protects against unresponsive or overloaded servers.

Ensures resources (memory, CPU) aren't held indefinitely.

Gives the user feedback instead of waiting forever.

How Timeout Works

SDKs let Client sets a per-request timeout (e.g., 30s).

If the deadline passes with no result → client triggers a timeout.

Client then sends a **cancellation notification** to tell the server to stop.

The server **must stop processing** that request and **not return a result**.

Progress Notification

Purpose of progress notification is to let the client know a long_running requests is still making progress.
Client includes a progress Token in the requests _meta.

========== Thank You ==========